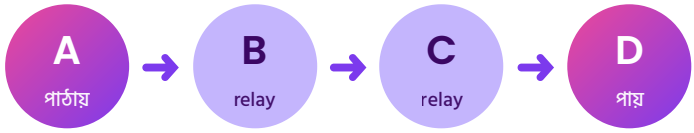
TB Chat Mesh

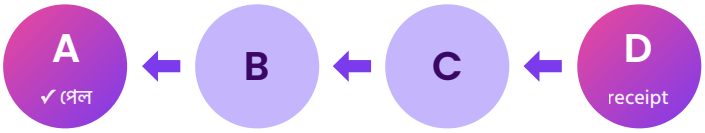
এক phone থেকে আরেক phone হয়ে message পৌঁছানো: idea আর কী কী লাগবে

Idea টা কী

এখন দুজন Nearby-তে একে অপরকে দেখে, pair করে, তারপর chat করে। নতুন idea হলো: A-এর message D-এর কাছে পৌঁছাবে, যদিও D সরাসরি A-এর কাছে নেই। মাঝের phone-গুলো (B, C) message টা হাত বদল করে পৌঁছে দেবে, কিন্তু ভেতরে কী লেখা আছে দেখতে পারবে না। D পেলে A-এর কাছে "D received" আসবে।



সিল করা খাম: B আর C শুধু "কাকে দিতে হবে" দেখে, ভেতরের লেখা দেখে না



"D received" receipt উল্টো পথে ফেরে, D-এর sign করা, তাই কেউ নকল করতে পারে না

> ✅ **এটা কি সম্ভব? হ্যাঁ**
>
> এই ধরনের network-কে **mesh network** বলে। Bridgefy, Briar-এর মতো offline chat app এভাবেই কাজ করে। আমাদের app-এ Google Nearby আর BLE আগে থেকেই আছে, ওর ওপরেই বানানো যাবে।

কী কী লাগবে

1 প্রত্যেক phone-এর নিজস্ব key · সবচেয়ে জরুরি

- এখন ID হলো ৮ অক্ষরের একটা random লেখা। এর জায়গায় প্রত্যেক phone-এর একটা key জোড়া থাকবে, public আর private (`cryptography` package)।
- ID হবে public key থেকে বানানো। এতে block-এর একটা পুরনো সমস্যাও মিটবে: এখন কেউ অন্যের ID নকল করতে পারে, তখন আর পারবে না।
- A-কে আগে থেকেই D-এর public key জানতে হবে: একবার সামনাসামনি দেখা হলে বা QR code scan করে।

2 Message-এর খাম (envelope)

```
{
  id:     "a91f…",      // একই message দুবার না নেওয়ার জন্য
  to:     "D-এর ID",    // relay শুধু এটুকু দেখে
  ttl:    5,            // প্রতি ধাপে ১ কমে, ০ হলে থামে
  sealed: "…"           // encrypted লেখা, শুধু D খুলতে পারে
}
```

3 নিজে থেকে connect (auto-connect)

- **Relay connection:** কাউকে না জিজ্ঞেস করে নিজে connect হবে, শুধু খাম পার করার জন্য। ভেতরের লেখা কেউ দেখে না, তাই accept লাগে না।
- **Chat pairing:** এখন যেমন আছে, মানুষ নিজে accept করবে।

4 পথ খোঁজা (routing)

- শুরুতে সবচেয়ে সহজ উপায় **flooding**: যার কাছে আসবে, সে তার সব connected phone-এ পাঠিয়ে দেবে।
- `id` আর `ttl` থাকায় message চক্রাকারে ঘুরতে থাকবে না।
- পরে চাইলে আরো বুদ্ধিমান routing যোগ করা যায়।

5 ধরে রেখে পরে পাঠানো (store and forward)

- B-এর কাছে পরের জন এখন না থাকলে B message কিছু সময় ধরে রাখবে, পরে কেউ এলে পাঠাবে।
- এখনকার pending message-এর ব্যবস্থাটাই এখানে কাজে লাগবে।

6 “D received” receipt

- D একটা receipt sign করে উল্টো পথে A-কে পাঠাবে। Sign করা, তাই মাঝের কেউ নকল receipt বানাতে পারবে না।

যেসব সীমাবদ্ধতা থাকবে

বিষয়	কী হবে
Phone-এর সংখ্যা	Nearby একসাথে মোটামুটি ৩ থেকে ৮টা phone-এর সাথে connect রাখতে পারে। বেশি মানুষ থাকলে network ছোট ছোট গুচ্ছে ভাগ হয়ে যায়।
App চালু রাখা	B আর C-কে relay করতে হলে app খোলা রাখতে হবে, অথবা background-এ চলতে হবে। Android-এ foreground service লাগবে, iPhone background-এ এটা প্রায় করতেই দেয় না।
Android ↔ iPhone	এর মধ্যে শুধু BLE চলে, যা খুব ধীর। ছোট text পাঠানো যাবে, ছবি কঠিন।
গোপনীয়তা	মাঝের phone লেখা দেখবে না, কিন্তু "কে কাকে পাঠাচ্ছে" দেখতে পাবে। এটাও লুকানো যায়, কিন্তু অনেক বেশি জটিল।
Spam	কেউ হাজার message পাঠিয়ে network ভরিয়ে দিতে পারে। প্রত্যেক phone থেকে কতগুলো যাবে তার সীমা দিতে হবে।
Block	Block করা মানুষের message relay হবে কিনা ঠিক করতে হবে। পরামর্শ: relay হোক, কারণ ভেতরে কী আছে কেউ দেখে না।
Battery	সবসময় scan আর connect চালু থাকলে battery বেশি খরচ হবে।

ধাপে ধাপে plan

ধাপ	কাজ	এর পরে কী দেখা যাবে
১	Key আর নতুন ID	প্রত্যেক phone-এর নকল-করা-যায়-না এমন ID। বাকি সব এর ওপর নির্ভর করে।
২	Auto relay connection (Android)	Flooding, `ttl`, আগে দেখা message মনে রাখা।
৩	Encrypted খাম আর receipt	A → B → C → D → "D received" পুরোটা কাজ করবে।
৪	Store and forward, background	পরের জন না থাকলেও message পরে পৌঁছাবে।
৫	iPhone (BLE)	iPhone-ও mesh-এ যোগ দেবে, ধীর গতিতে।

প্রথম তিনটা ধাপ হলেই চারটা Android phone বা emulator দিয়ে A → B → C → D আর receipt পুরোটা দেখানো যাবে।